Koban — Information Security Policy

Owner: Vishhvak Srinivasan (sole operator and security contact) **Effective:** 2026-07-06 · **Review cadence:** every 6 months or after any material architecture change **Scope:** the Koban personal-finance application (this repository), its production deployment on Railway (`finances.vishhvak.com`), the production PostgreSQL database, connected data-provider accounts (Plaid, Teller, SnapTrade, SimpleFIN), and the operator workstation used for development.

Koban is a single-operator application: the only end user is the operator. There are no employees, contractors, or third-party users. Controls below are sized to that reality and stated as practiced facts, not aspirations.

1. Governance

- The operator is accountable for all security decisions, reviews this policy on the stated cadence, and remediates identified gaps.
- Security-relevant changes (auth, data handling, providers, infrastructure) are made through version-controlled code changes in this repository, giving a complete audit history via git.

2. Access control

- **Production assets** (Railway project, production database, DNS, GitHub repository, provider dashboards) are accessible only to the operator's individual accounts. No shared accounts exist.
- All administrative accounts (GitHub, Railway, Plaid, Teller, SnapTrade, SimpleFIN, domain registrar) require multi-factor authentication.
- **Application access (hosted mode):** the application enforces authentication via Better Auth when `DATABASE_URL` is set. Registration is closed after the first (owner) account. Passkey (WebAuthn, phishing-resistant) and TOTP second factors are supported and enrolled; password-only sign-in does not grant a session when a second factor is enrolled.
- API access requires a Better Auth session, a Better Auth API key, or the instance bearer token; unauthenticated requests to `/api/*` are rejected in hosted mode.
- Access review: because exactly one identity exists per system, access review consists of verifying (at each policy review) that no additional accounts, keys, or OAuth grants have been added; stale provider tokens and API keys are revoked when found.

3. Secrets management

- Secrets (provider tokens, API keys, database credentials) live in the git-ignored `.env` locally and in Railway service variables in production. They are never committed to version control; `.env.example` carries variable names only.
- Long-lived provider secrets that are ever exposed outside these stores (e.g., pasted into a chat or terminal shared with a third-party tool) are treated as compromised and rotated.
- Provider connections are read-only by construction (Plaid Link read scopes, Teller, SnapTrade read connections, SimpleFIN read-only protocol); no money-movement capability is provisioned.

4. Encryption

- **In transit:** all production traffic is HTTPS/TLS (Railway edge, TLS 1.2+; certificates managed by Railway). Provider API calls are HTTPS only.
- **At rest (production):** the production PostgreSQL database runs on Railway, a SOC 2 Type II certified platform whose storage is encrypted at rest (see trust.railway.com).
- **At rest (workstation):** the development machine uses full-disk encryption (macOS FileVault, verified enabled).

5. Vulnerability and patch management

- Dependencies are monitored via GitHub Dependabot (weekly) and `pnpm audit` in CI on every push.
- **Patch SLA:** critical/high advisories affecting production code paths are patched within 7 days; others at the next regular dependency update. Framework majors are tracked and upgraded deliberately (e.g., Next.js is kept on the current stable major).
- The operator workstation runs macOS with automatic security updates enabled.
- End-of-life software: the stack (Node LTS, Next.js current stable, PostgreSQL 17) is reviewed at each policy review; EOL components are upgraded.

6. Data protection, retention, and deletion

- Data stored: financial account metadata, balances, and transactions retrieved read-only from institutions the operator explicitly connected, plus operator-authored annotations (categories, tags, notes).
- Data is retained while the relevant provider connection remains active, to preserve ledger history.

- Deletion: the operator can delete all application data at any time via the built-in local-data reset (Settings), and can revoke provider access at each provider (Plaid Portal, Teller, SnapTrade dashboard, SimpleFIN Bridge). Railway volumes/databases are deleted when the hosting project is deleted.
- No consumer data is sold or shared. Data leaves the system only toward the connected providers themselves or, if the operator explicitly enables AI enrichment, the operator-configured LLM endpoint (disabled by default).
- The privacy policy shown in-app (`/privacy`) reflects this section and is kept in sync with it.

7. Application security practices

- All changes pass typechecking, unit tests, a production build, and end-to-end tests before being considered done (enforced convention; also run in CI).
- API routes validate inputs and return deliberate 400s on malformed bodies; provider data is treated as data, never executed.
- The database write path is a single serialized boundary (`lib/db/finance-db.ts`), limiting injection and race surface.
- No fake or fabricated financial data exists in runtime paths; provider readiness is reported honestly.

8. Incident response (right-sized)

On suspected compromise of any credential, token, or the database: (1) revoke/rotate the affected credential at its source; (2) disconnect affected provider items; (3) inspect git history, Railway deploy/access logs, and provider dashboards for unauthorized activity; (4) restore data from provider re-sync (all financial data is re-derivable from providers); (5) record the incident and remediation as a dated note in this directory.

9. Review log

- 2026-07-06 — Initial policy authored alongside MFA (passkey + TOTP), privacy policy page, consent copy, Dependabot, and CI introduction.